

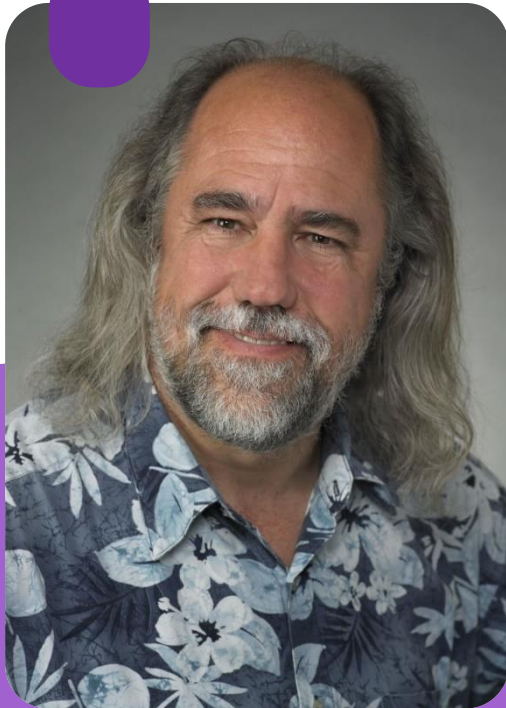


UNIFIED MODELING LANGUAGE

Standard terminologies and
diagram conventions for object-
oriented

The Three Amigos

Object Management Group



Grady Booch



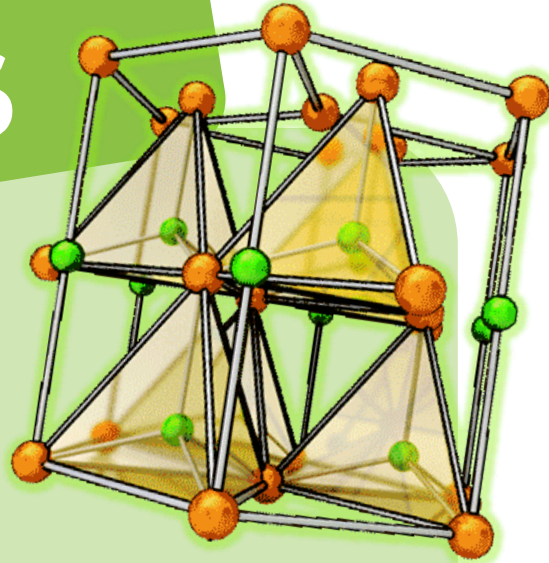
Jim Rumbaugh



Ivar Jacobson

STRUCTURAL DIAGRAMS

- Class
- Package
- Object
- Component
- Composite Structure



- Use Case
- Sequence
- Activity
- State



BEHAVIORAL DIAGRAMS

Class Diagrams

Models the static structure of a system.

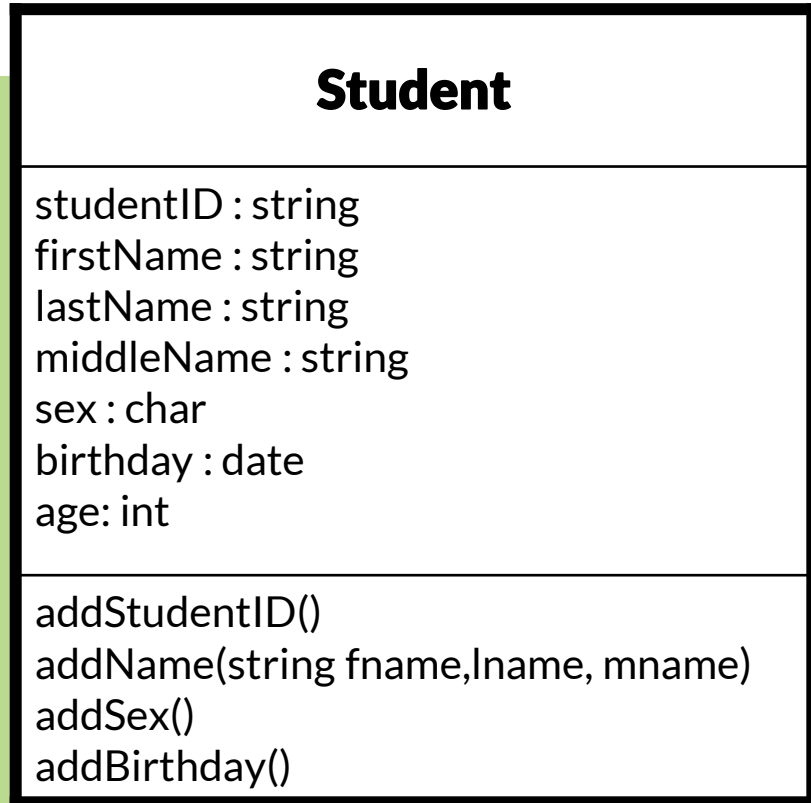
It shows relationship among classes, object, attributes and operations.



Student

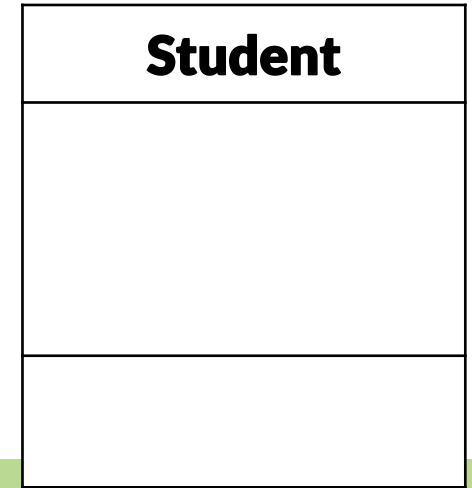
Student
studentID : string firstName : string lastName : string middleName : string sex : char birthday : date age: int
addStudentID() addName(string fname,lname, mname) addSex() addBirthday()

Class Diagrams



Passive Classes

Store data and serve other classes



Active Classes

Initiate and control the flow of activity

Class Diagrams

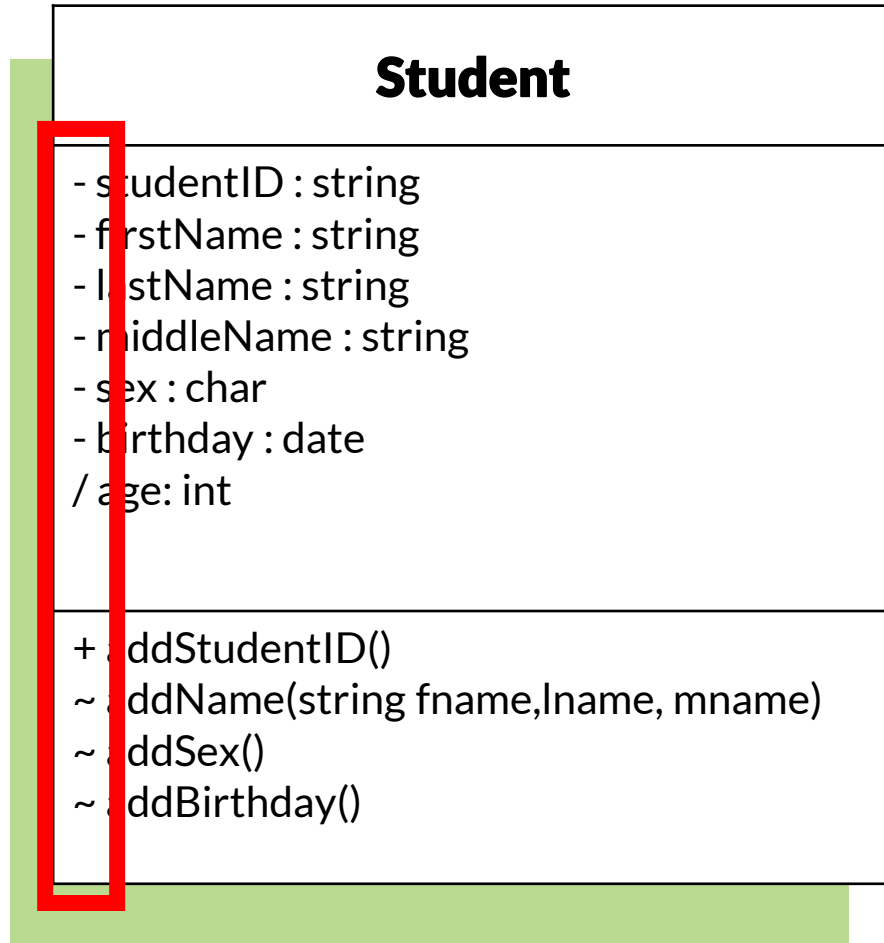
Symbol	Description
+	public
-	private
#	protected
/	derived
~	package

Visibility Makers

Signify the accessibility of the information within the class

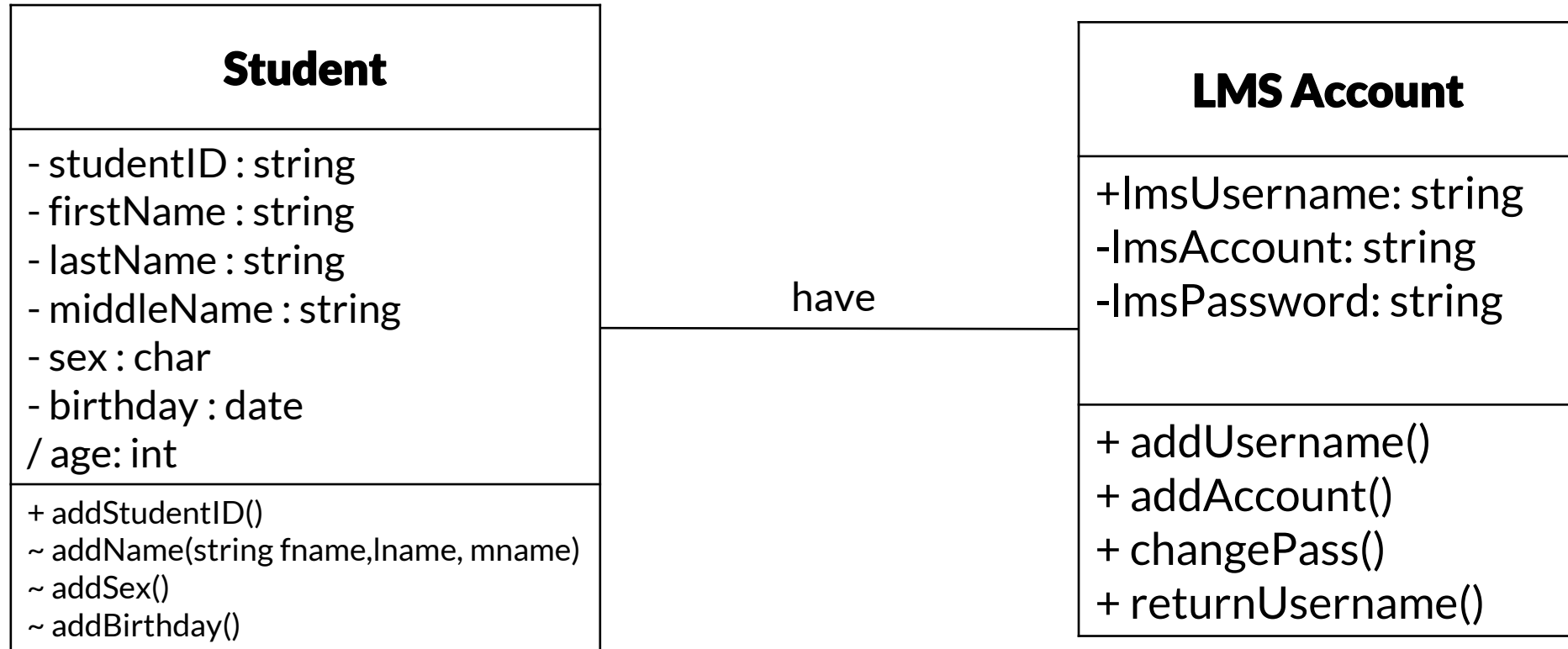
Applies in attributes and operations.

Class Diagrams



Symbol	Description
+	public
-	private
#	protected
/	derived
~	package

Class Diagrams(Relationship Type)



Static relationship between classes

Association

Class Diagrams

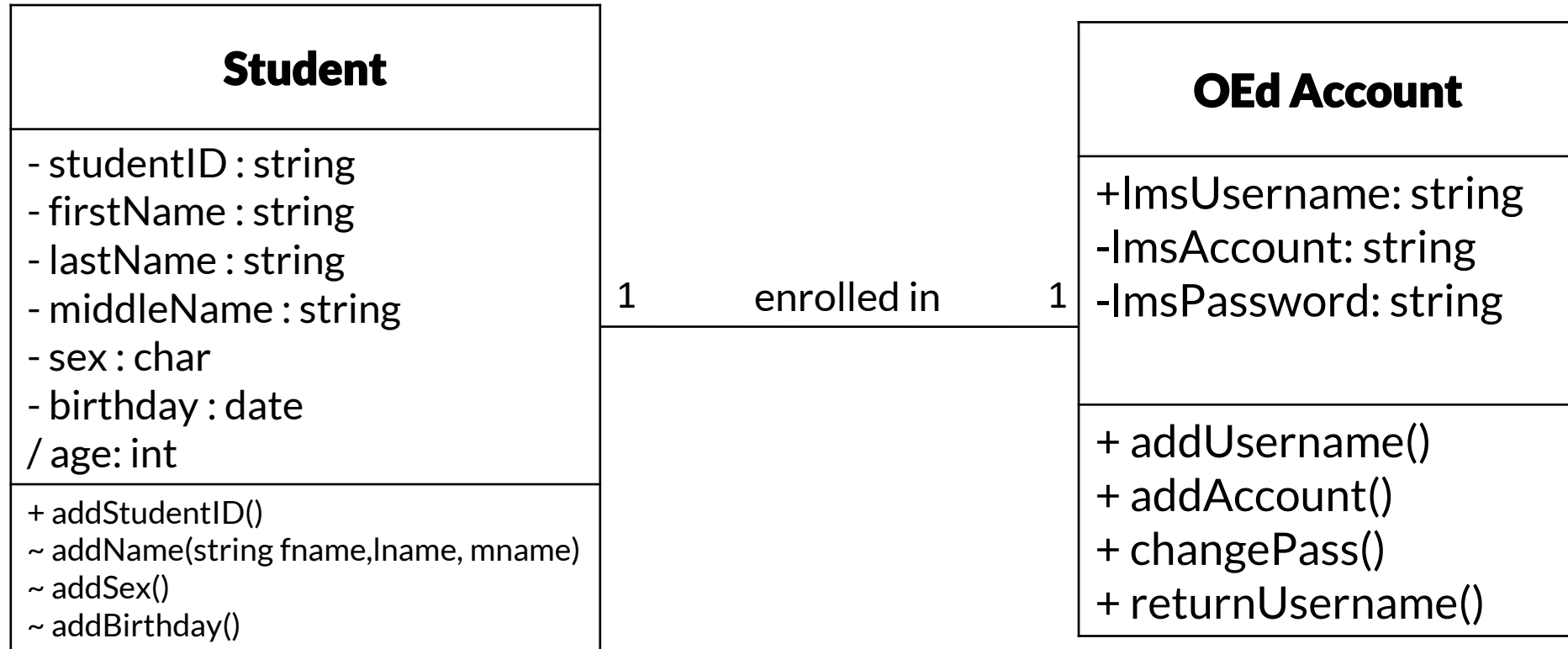
Indicator	Meaning
1	one only
0..1	zero or 1
*	many
1..*	one or many
0..*	zero or many

Indicator	Meaning
n	declared number
0..n	zero or <n
1..n	1 or <n

Indicates number of instance of one class linked to an instance of another class

Multiplicity

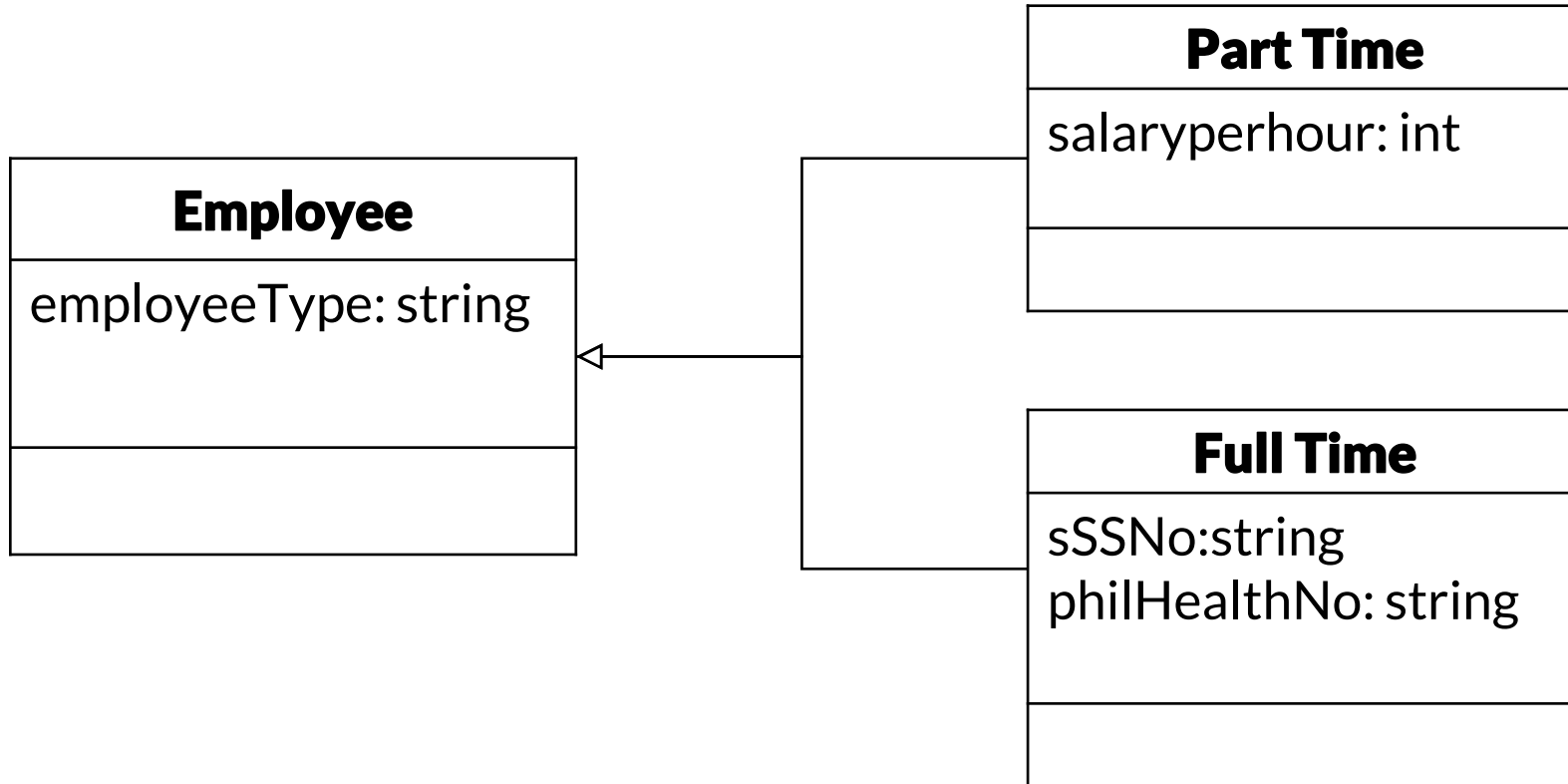
Class Diagrams



Indicates number of instance of one class linked
to an instance of another class

Multiplicity

Class Diagrams (Relationship Type)



Also called Inheritance, a specialized version of another class

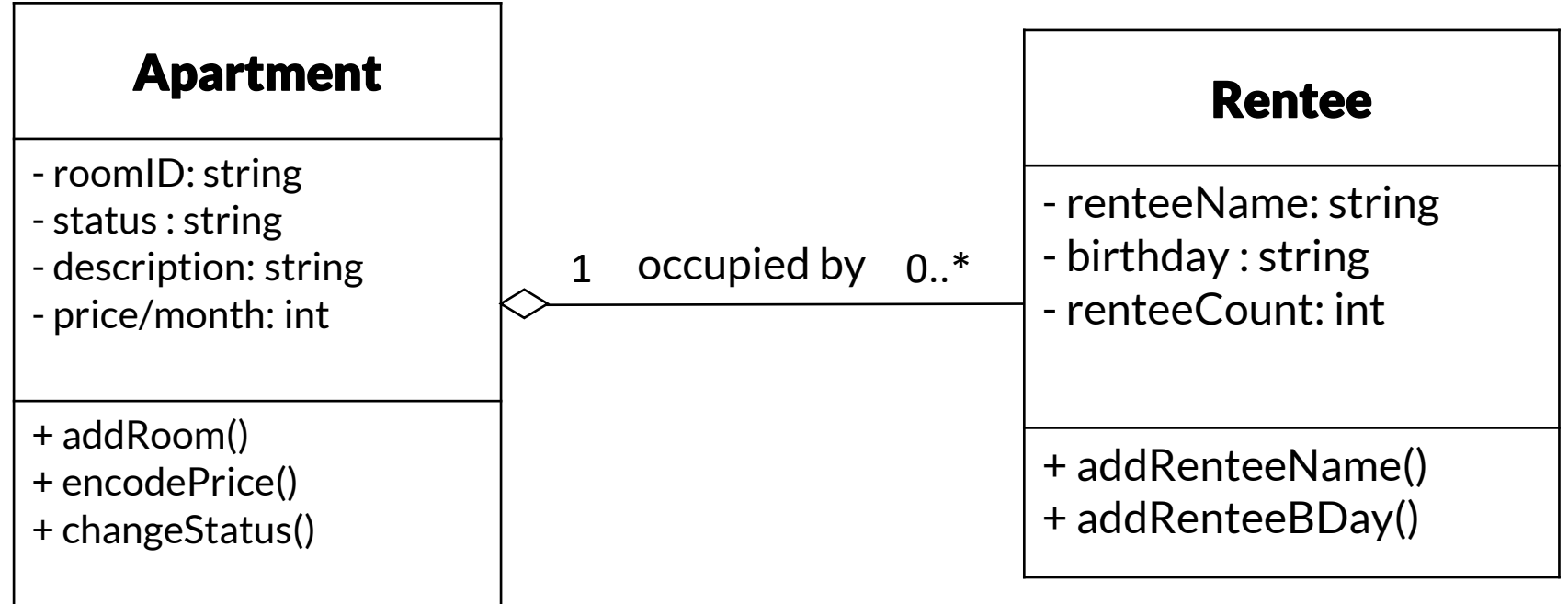
Generalization

Class Diagrams(Relationship Type)

Explanation:

A class ROOM APARTMENT and RENTEE are different independent class.

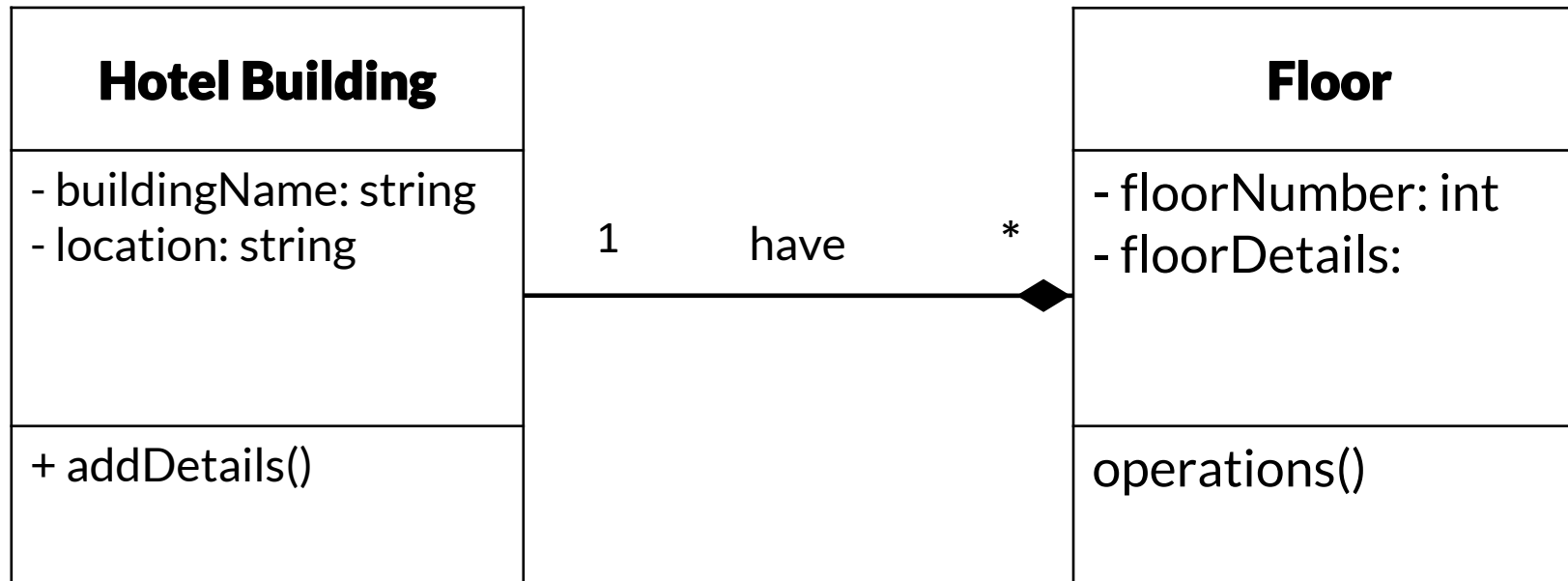
The relation is aggregation because the room apartment may be occupied by Rentee, Rentee can come and go.



Show how classes is compose of other classes, Independent

Aggregation

Class Diagrams (Relationship Type)



Strong ownership between whole and its part

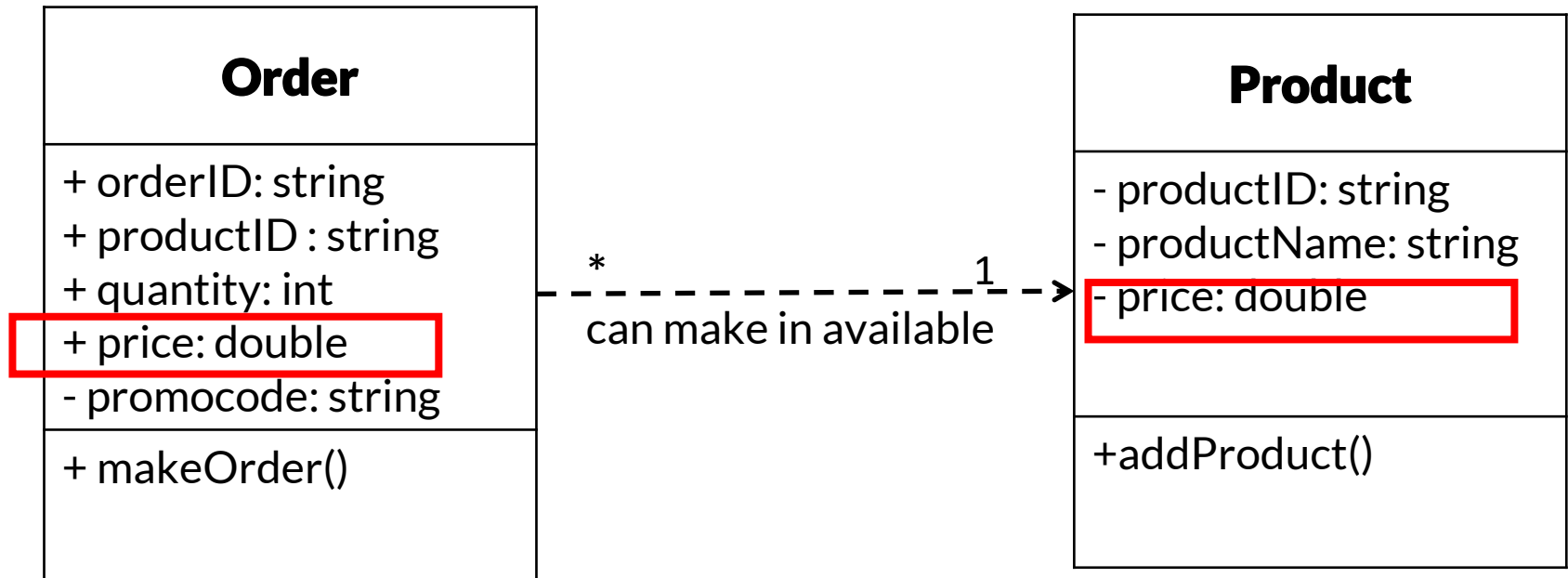
Composition

Class Diagrams (Relationship Type)

Explanation:

The price in the Order is dependent on the Product.

The Product price can change the future Order price but a change in Order price will not affect the Product price.



the changes to the definition of one may cause changes to the other but not the other way around

Dependency

Object Diagrams



Represents an instance of a particular moment.

Object relationship of a system.

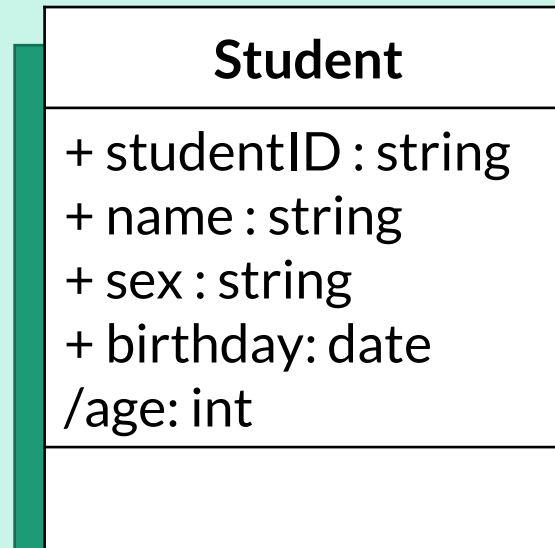
Focus on the Functionalities

Object Name: Classifier Name
object attributes: value

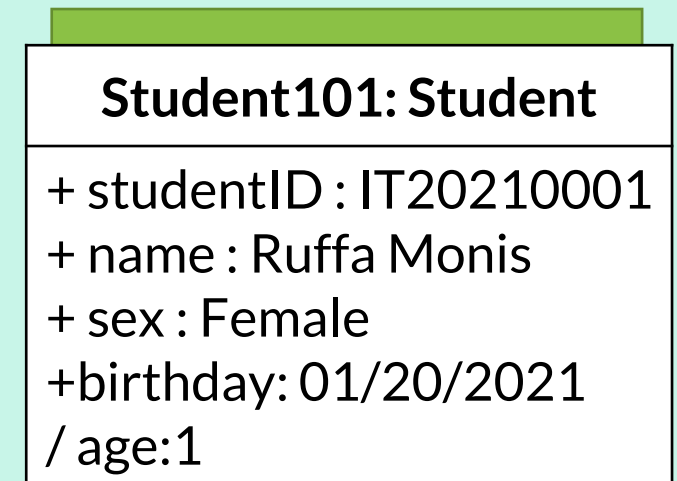
Object Diagrams

- Show connection between instance, value, slots and links.
- It verifies the completeness and accuracy of class diagram.

Class Diagram

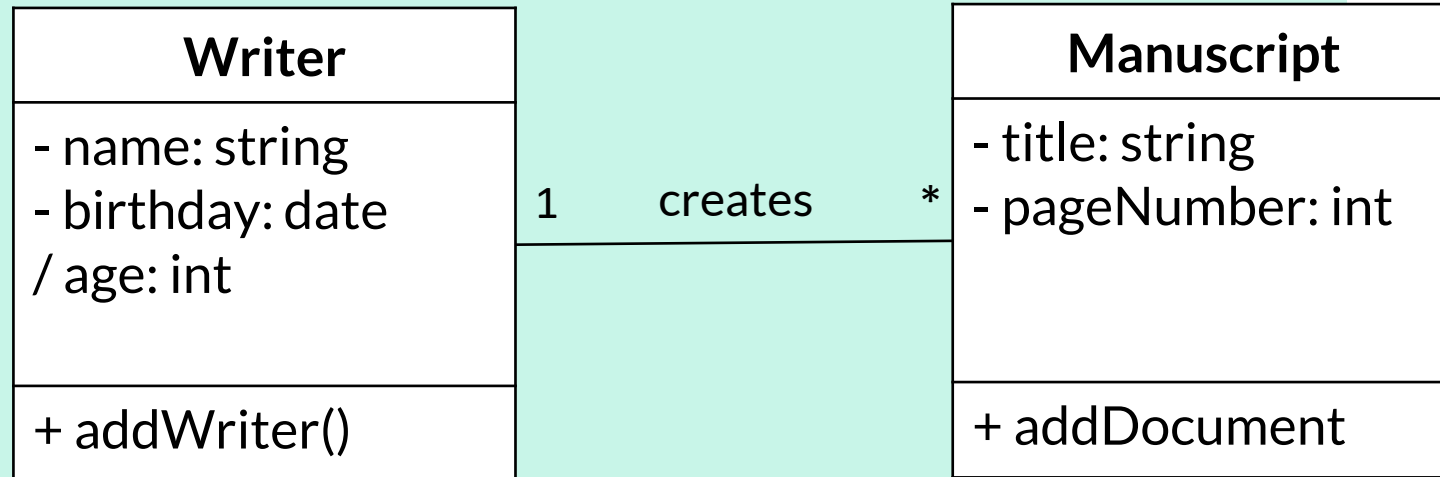


Object Diagram



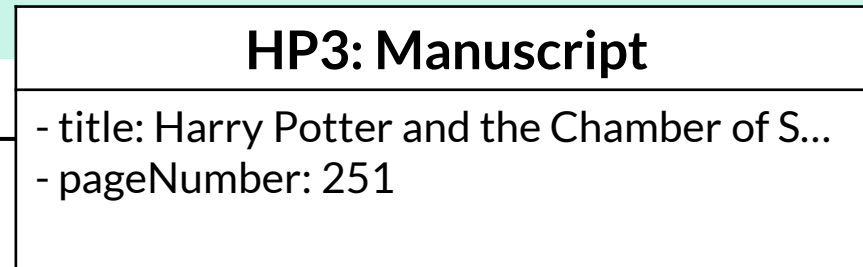
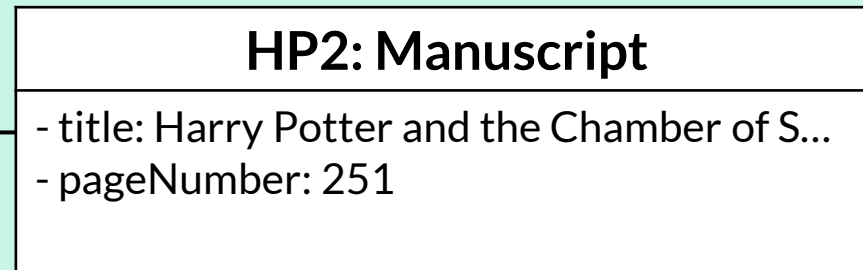
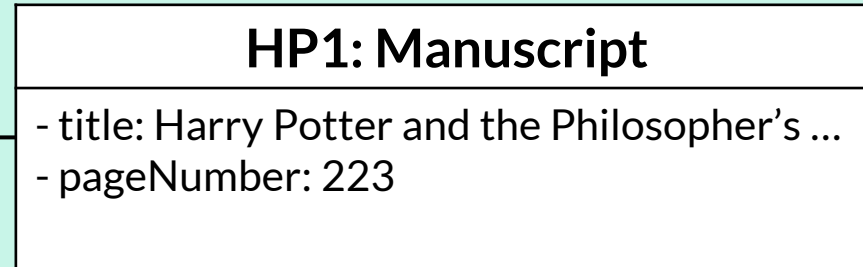
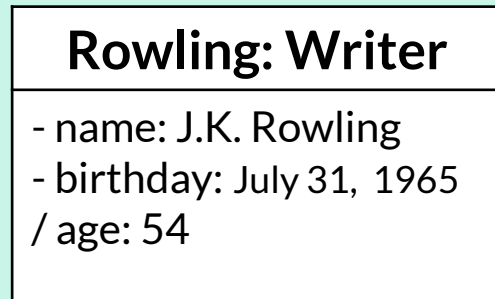
Object Diagrams

Class Diagram



Object Diagrams

Object Diagram



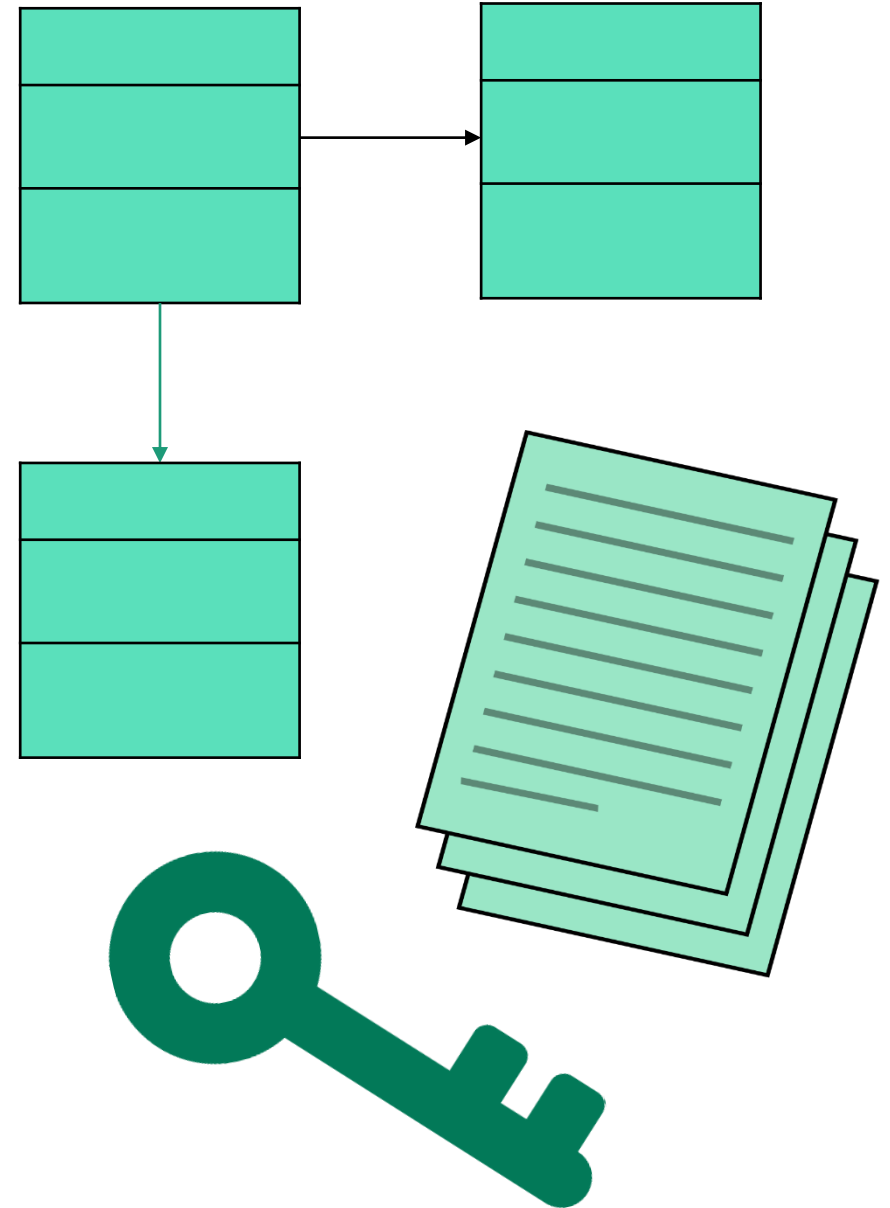
Object Diagrams

Class	Object
It represents static aspects of a system.	It represents the behavior of a system in real time.
It doesn't include dynamic changes.	It captures runtime changes of a system.
It never includes attributes or data values of an instance.	It includes attributes and data values of any instance.
Class diagram manipulates the behavior of objects.	Objects are instances of classes.

Package Diagrams



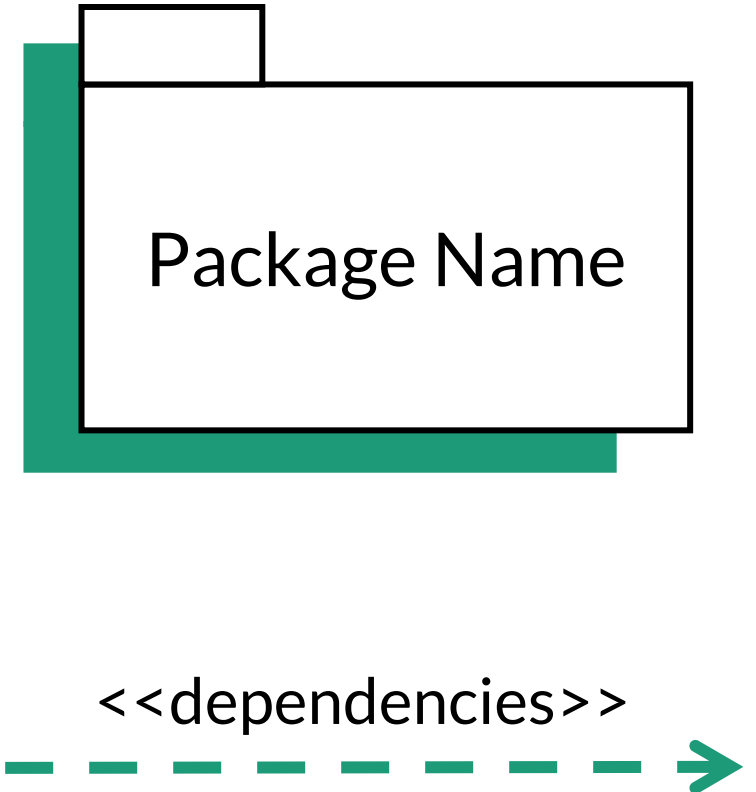
- Organizing model for middle to large scale system.
- Package contains classes, access and document.



Package Diagrams

 Simplify complex class diagrams.

 Shows structures and dependencies

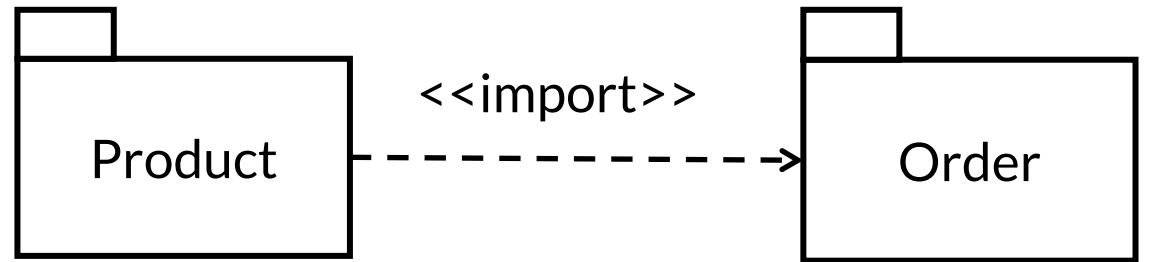


Package Diagrams

<<import>>

one package imports the
functionality of other package.

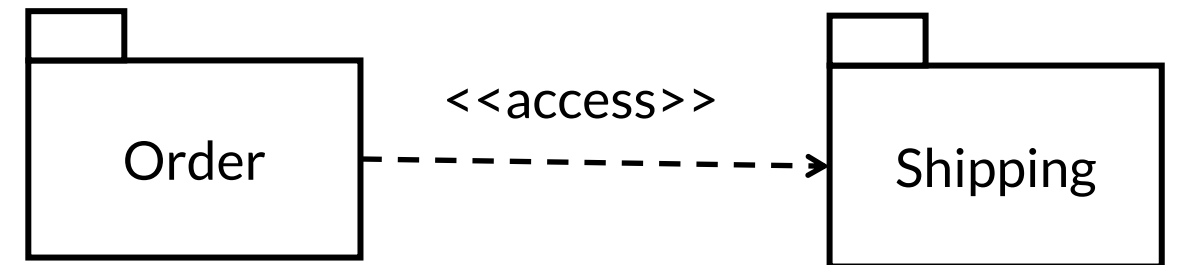
SOURCE ADDED TO TARGET PACKAGE



<<access>>

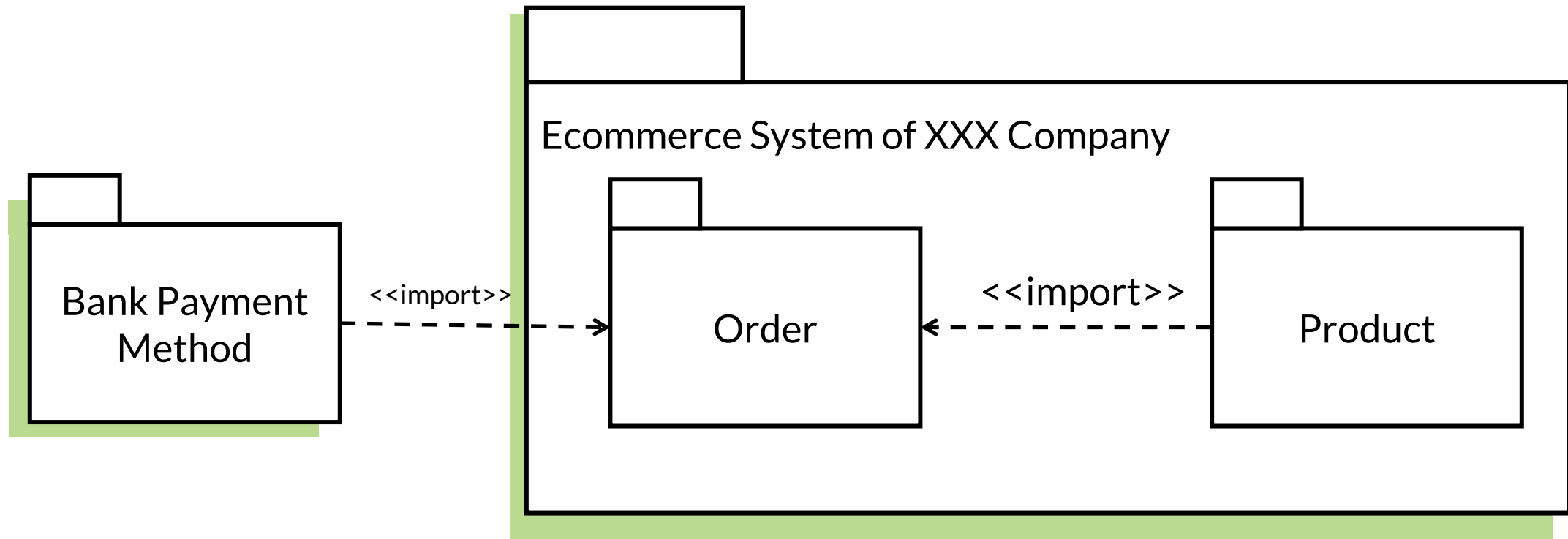
one package requires help from
functions of other package.

SOURCE AVAILABLE TO TARGET PACKAGE



Dependency Notation

Package Diagrams

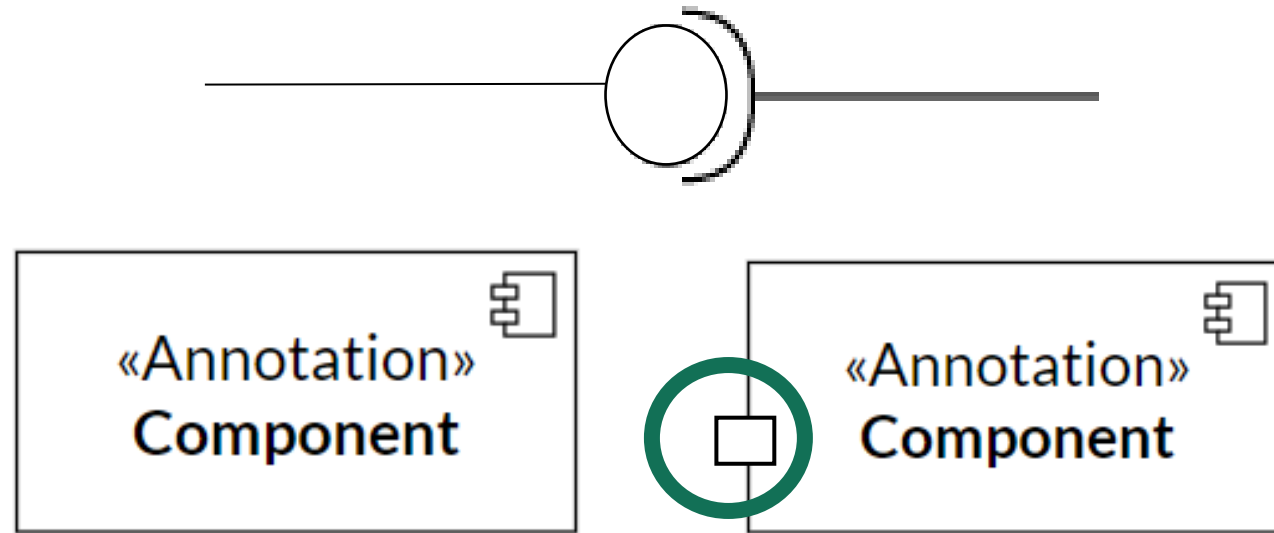


Component Diagrams

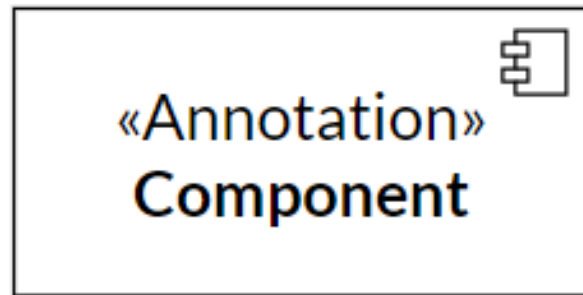


- Modeling the physical aspects of object-oriented systems.
- Responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.

Symbols of Component Diagrams



Component Diagrams



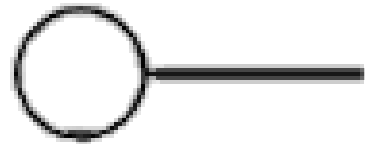
Component Symbol

It represents a **modular part of a system** that encapsulates its contents and whose manifestation is replaceable within its environment

Component Diagrams



Interface Symbol



Provided Interface

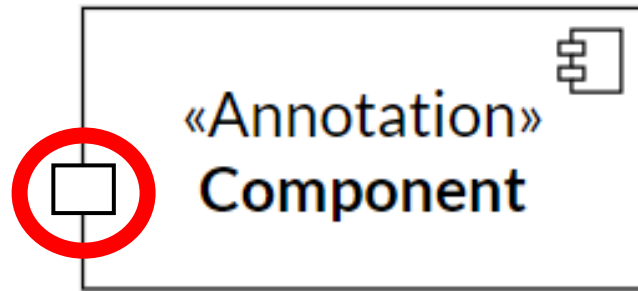
interfaces where a component produces information used by the required interface of another component.



Required Interface

interfaces where a component requires information in order to perform its proper function.

Component Diagrams

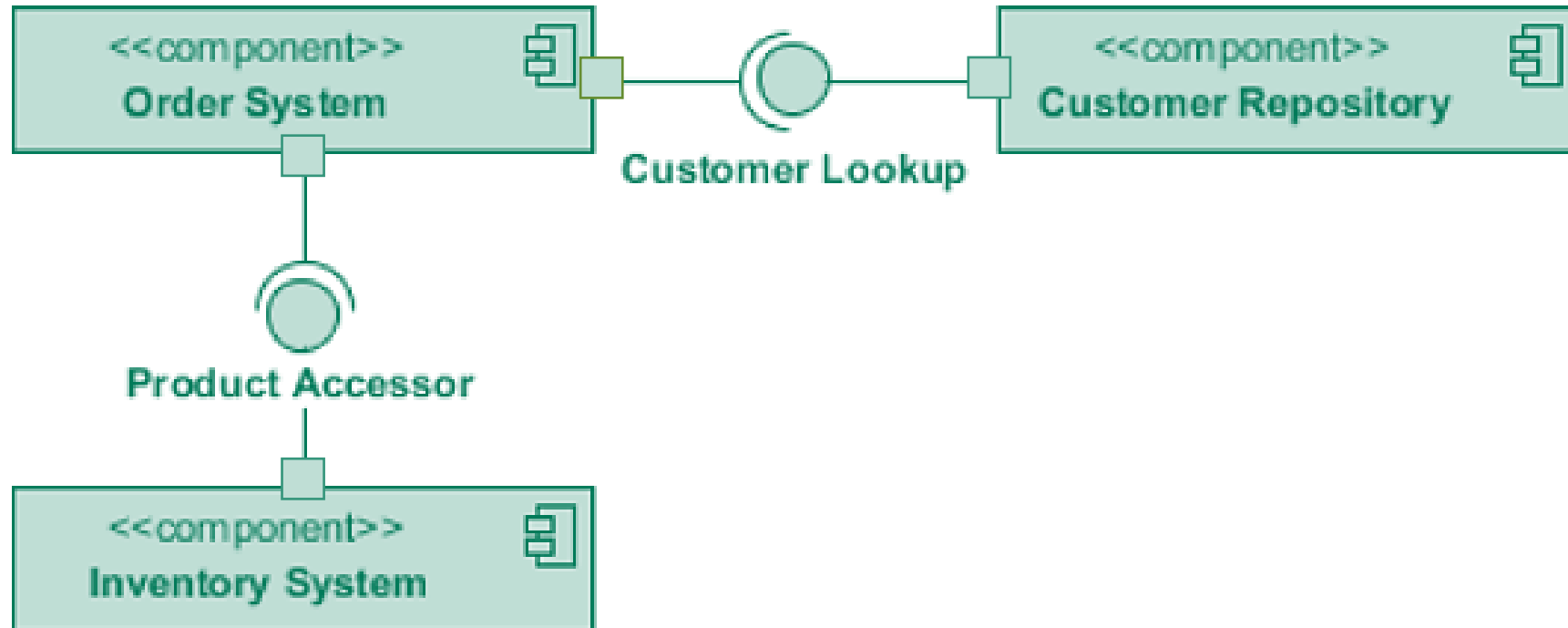


Port Symbol

It represented using a square along the edge of the system or a component.

help expose required and provided interfaces of a component.

Component Diagrams



Component Diagrams

